



MANIPAL INSTITUTE OF TECHNOLOGY
MANIPAL
(A constituent unit of MAHE, Manipal)

Report On

Design and Implementation of an Information Retrieval System

Submitted To

Dr. Radhika M Pai

Department of Computer Science & Engineering

Submitted By

Anusha A Naik (251090130032)

II semester, M.Tech CSE

Table of Contents

Chapter	Title	Page No.
1	Abstract	1
2	Introduction	1
3	Dataset Description	1
4	Methodology	1-2
4.1	Text Preprocessing and Indexing	1
4.2	Tolerant Retrieval	1
4.3	Index Compression	2
4.4	Term Weighting and Vector Space Model	2
4.5	Scoring and Complete Search System	2
4.6	Relevance Feedback via Rocchio Algorithm	2
5	System Implementation	3
6	Results	3-4
6.1	Spelling Correction	3
6.2	TF-IDF and cosine similarity ranking	3
6.3	Rocchio Query Expansion	4
7	Conclusion	4
8	References	4

1. Abstract

This project details the end-to-end development of a custom Information Retrieval (IR) system from scratch. The objective was to practically implement the core algorithms of IR, spanning from basic Boolean indexing to advanced Vector Space ranking and query expansion. The system was built using Python and evaluated on the standard Cranfield dataset collection. Key features include text preprocessing, Levenshtein based spelling correction, Variable-Byte index compression, TF-IDF scoring with Cosine similarity, and Rocchio Pseudo-Relevance Feedback. Finally, the engine was deployed via a FastAPI backend to provide an interactive graphical user interface.

2. Introduction

Information Retrieval (IR) is the science of searching for documents, information within documents, and metadata about those documents. Modern search engines rely on complex pipelines to ensure both high-speed retrieval and high-quality ranking. The goal of this project is to demonstrate a deep practical understanding of fundamental IR concepts by building a complete search engine.

3. Dataset Description

To accurately build and evaluate the system, the Cranfield Collection was used. This is a pioneering IR test collection consisting of:

- 1,400 documents: Scientific abstracts related to aerodynamics.
- 225 test queries: Standardized user search queries.
- Relevance Judgments: A ground-truth mapping of which documents are mathematically relevant to which queries, enabling rigorous system evaluation.

4. Methodology

The system architecture was developed sequentially, corresponding to standard IR processing stages.

4.1. Text Preprocessing and Indexing

Raw text cannot be efficiently searched. The dataset was passed through a Natural Language Processing (NLP) pipeline using the NLTK library:

1. Tokenization & Case Folding: Converting text to lowercase and splitting it into tokens.
2. Stopword Removal: Eliminating non-informative words (e.g., "the", "is", "at").
3. Stemming: Applying the Porter Stemmer algorithm to reduce terms to their morphological roots (e.g., "aerodynamics" to "aerodynam").

An Inverted Index was then constructed, mapping each of the 4,391 unique dictionary terms to a sorted list of Document IDs.

Code Snippet 1: Preprocessing Pipeline

```
def preprocess_text(text):  
    tokens = word_tokenize(text.lower())  
    return [stemmer.stem(w) for w in tokens if w.isalnum() and w not in stop_words]
```

4.2. Tolerant Retrieval

To handle user typos and misspelled queries, the system implements Tolerant Retrieval using the Levenshtein Edit Distance algorithm. If a query term is not present in the dictionary, a dynamic programming matrix calculates the minimum edits (insertions, deletions, substitutions) required to match an existing dictionary term, automatically suggesting the correction.

Code Snippet 2: Levenshtein Distance Matrix

```
def levenshtein_distance(s1, s2):
    m, n = len(s1), len(s2)
    dp = [[0] * (n + 1) for _ in range(m + 1)]
    for i in range(m + 1): dp[i][0] = i
    for j in range(n + 1): dp[0][j] = j
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            cost = 0 if s1[i - 1] == s2[j - 1] else 1
            dp[i][j] = min(dp[i - 1][j] + 1, dp[i][j - 1] + 1, dp[i - 1][j - 1] + cost)
    return dp[m][n]
```

4.3. Index Compression

To minimize memory footprint, the postings lists were compressed. Absolute Document IDs were first converted using Delta Encoding to represent the numerical gaps between IDs. As gap integers are generally small, Variable Byte (V-Byte) Encoding was applied, compressing standard 32-bit integers into 8-bit bytes using a continuation bit. Testing on high-frequency terms demonstrated a 75% reduction in memory storage.

4.4. Term Weighting & Vector Space Model

To transition from exact Boolean matches to Ranked Retrieval, the system utilizes the Vector Space Model (VSM).

- TF-IDF Weighting: Terms were weighted using Log-frequency Term Frequency and Inverse Document Frequency.
- Every document and user query is represented as a high-dimensional mathematical vector based on these term weights.

4.5. Scoring and Complete Search System

Documents are ranked based on their Cosine Similarity to the user's query vector. To optimize retrieval speed, Index Elimination is used. The system calculates the dot product only for documents that contain at least one term present in the query.

4.6. Relevance Feedback via Rocchio Algorithm

To improve the system's Recall, Pseudo-Relevance Feedback was integrated. When enabled, the system assumes the top-retrieved documents are highly relevant. It calculates the vector centroid of these documents, extracts the highest-weighted missing terms, and adds them to the original query.

Code Snippet 3: Rocchio Expansion Logic

```
# Calculate centroid of pseudo-relevant documents
for d in rel_docs:
    for t, w in doc_vectors[d].items():
        centroid[t] = centroid.get(t, 0.0) + float(w)
for t in centroid:
    centroid[t] = float((centroid[t] / len(rel_docs)) * 0.75)
# Extract top terms to append to the user's query
sorted_new_terms = sorted(candidate_new_terms.items(), key=lambda x: x[1], reverse=True)[:2]
rochio_terms = [str(t[0]) for t in sorted_new_terms]
```

5. System Implementation

The core IR algorithms were wrapped in a FastAPI web framework. A Jinja2 HTML template was used to create an interactive, web-based search interface, allowing for real-time querying, pagination (top 10 results out of all matches), and dynamic toggling of the Rocchio algorithm. The system's retrieval effectiveness was analysed using standard IR metrics Precision and Recall. As Precision and Recall share an inverse trade-off, the system relies on Mean Average Precision (MAP), a unified metric that calculates the average precision across all recall levels for all 225 Cranfield queries.

6. Results

6.1. Spelling Correction Demonstration

When a user inputs a query with a typographical error, the system successfully intercepts it, calculates the Levenshtein distance, and proceeds with the corrected query.

Cranfield Search Engine

expeiment

☐ Enable Rocchio Query Expansion

*Did you mean: **experiment?***

Showing top 10 out of **337** matching documents.

an experimental test of compressibility transformation for turbulent boundary layer .

Doc ID: 271 | Cosine Score: 0.1502

an experimental test of compressibility transformation for turbulent boundary layer . discussion of various turbulent-boundary-layer theories, in the light of experimental measurements by matting an...

the bending strength of pressurized cylinders .

Doc ID: 1045 | Cosine Score: 0.1428

the bending strength of pressurized cylinders . discussion of previously presented experimental data for the loading of pressurized cylinders, in terms of membrane theory

Fig 6.1 spelling correction

6.2. TF-IDF and Cosine Similarity Ranking

The baseline Vector Space Model accurately ranks documents. The stemmer successfully groups variations of words (e.g., searching "fly" retrieves documents containing "flight").

Cranfield Search Engine

fly

☐ Enable Rocchio Query Expansion

Showing top 10 out of **16** matching documents.

two and three-dimensional unsteady lift problems in high speed flight .

Doc ID: 700 | Cosine Score: 0.2592

two and three-dimensional unsteady lift problems in high speed flight . the problem of transient lift on two- and three-dimensional wings flying at high speeds is discussed as a boundary-value proble...

on the ground level disturbance from large aircraft flying at supersonic speeds .

Doc ID: 253 | Cosine Score: 0.2111

on the ground level disturbance from large aircraft flying at supersonic speeds . the whitam-walkden theory for the estimation of the strength of shock waves at ground level from aircraft flying at...

the estimation of fatigue damage on structural elements .

Doc ID: 884 | Cosine Score: 0.2042

the estimation of fatigue damage on structural elements . a method is presented for the estimation of fatigue damage to aircraft structural elements . the gust spectrum to which the aircraft is sub...

Fig 6.2: document ranking

6.3. Rocchio Query Expansion

When Pseudo-Relevance Feedback is enabled, the system successfully extracts domain-specific terminology from the initial results and expands the user's search, effectively increasing Recall.

Cranfield Search Engine

fly

☒ Enable Rocchio Query Expansion
Rocchio automatically added: *flight, aircraft*

Showing top 10 out of **1398** matching documents.

on trails of axisymmetric hypersonic blunt bodies flying through the atmosphere .
Doc ID: 85 | Cosine Score: 0.4392
on trails of axisymmetric hypersonic blunt bodies flying through the atmosphere . the trail left in the atmosphere by a body moving at hypersonic speeds is the subject of theoretical treatment . th...

some effects of bluntness on boundary layer transition and heat transfer at supersonic speeds .
Doc ID: 1300 | Cosine Score: 0.4219
some effects of bluntness on boundary layer transition and heat transfer at supersonic speeds . large downstream movements of transition observed when the leading edge of a hollow cylinder or a flat...

Fig 6.3 Rocchio Query Expansion

8. Conclusion

The fundamental concepts of information retrieval were effectively put together in this work. By integrating NLTK preprocessing, it demonstrated the mathematical feasibility of the Vector Space Model, the memory efficiency of V-Byte encoding, and the accuracy improvements provided by Rocchio Relevance Feedback. The deployment of the engine via FastAPI demonstrates that the theoretical concepts covered can be successfully combined to build a modern, production-ready search architecture.

References

1. C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge, U.K.: Cambridge University Press, 2009.
2. "Cranfield experiments," *Wikipedia*. [Online]. Available: https://en.wikipedia.org/wiki/Cranfield_experiments
3. J. Zobel and A. Moffat, "Inverted files for text search engines," *ACM Computing Surveys (CSUR)* *ACM Computing Surveys (CSUR)*, vol. 38, no. 2, pp. 6–es, 2006.
4. S. Bird, E. Klein, and E. Loper, *Natural Language Processing with Python*, O'Reilly Media, 2009.
5. "Mean Average Precision (mAP) Explained," *Built In*, 2024. [Online]. Available: <https://builtin.com/articles/mean-average-precision>.